

リストをあらためて

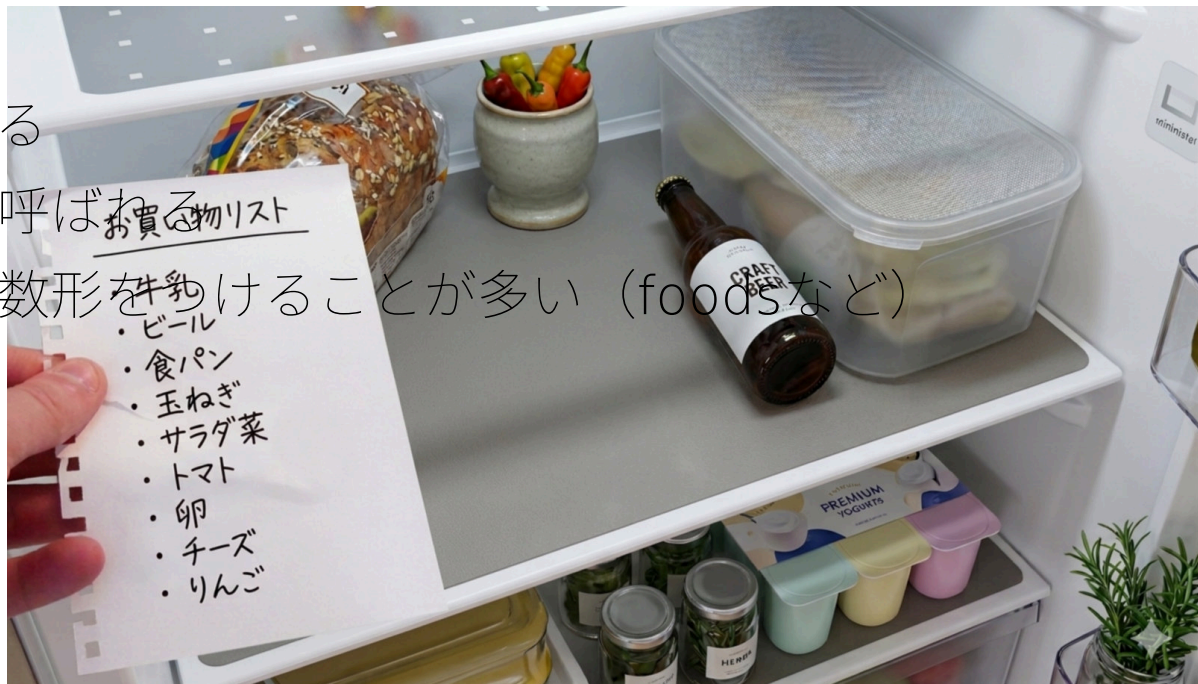
少しだけ詳しく

rkskmt

1

復習(listとは)

- 最もよく使われるコンテナ
- データは追加順に並んでいる
- 他言語では配列やアレイと呼ばれる
- 変数名は、要素の名詞の複数形を付けることが多い (foodsなど)



2

宣言

`[]` でカンマ区切りでデータを入力するとリストになる。

- 順序が保持される
- 重複してもよい

```
1 scores = [85, 90, 92, 78, 92, 88]
2 print(scores) # [85, 90, 92, 78, 92, 88]
```

Python

💡 お作法

カンマ `,` の後は**必ずスペースを空ける** のが綺麗に書くコツ。

3

アクセス（インデックス）

要素にアクセスするには**インデックス**（数字）を使う。

- インデックスは**ゼロ始まり**
- 後ろにいくにつれて 1 ずつ増加

```
1 scores = [85, 90, 92, 78, 92, 88]
2 print(scores[0]) # 85
3 print(scores[4]) # 92
```

Python

4

マイナスインデックス

インデックスをマイナスにすると **末尾からアクセス** できる。

- **-1** が末尾（**-0** ではない）

```
1 print(scores[-1]) # 88 (末尾)
2 print(scores[-5]) # 90 (末尾から5番目)
```

Python

5

要素の追加（append）

append メソッドで末尾に要素を追加できる。

```
1 scores = [85, 90, 92, 78, 92, 88]
2 scores.append(95)
3 print(scores) # [85, 90, 92, 78, 92, 88, 95]
```

Python

⚠ 戻り値は **None**

append は末尾に追加するだけで、**新しいリストを返さない**。

```
1 result = scores.append(95)
2 print(result) # None ← リストではない！
```

Python

result = scores.append(95) のように書くと **result** が **None** になってしまう。

6

走査（全要素へのアクセス）

リストの全要素にアクセスすることを **走査** という。 **for** 文で簡単に書ける。

```
1 for score in scores:
2     print(f"{score}点")
```

Python

85点
90点
92点
78点
92点
88点

7

要素の存在確認（**in** 演算子）

in 演算子でリストに特定の値が含まれているか確認できる。

```
1 scores = [85, 90, 78]
2
3 if 90 in scores:
4     print("90点の学生がいる") # 90点がいる
5
6 if 100 not in scores:
7     print("満点はいない") # 満点はいない
```

💡 **in** vs インデックス検索

「含まれているか知りたいだけ」なら **in** が最もシンプル。位置（インデックス）も必要なときは **.index()** を使う。

```
1 print(scores.index(90)) # 1
```

8

listを処理する便利なアレコレ

- スライス（部分集合の取得）
- 2次元配列
- N次元配列
- list 同士の結合
- sortとsorted
- 内包表記



9

範囲外のインデックスはエラー

以下のインデックスは指定できない（エラーになる）。

- 要素数（list の長さ）以上
- マイナス要素数+1 以下

```
1 test = [0, 1, 2, 3]
2 print(len(test)) # 4
3 print(test[4]) # IndexError
4 print(test[-5]) # IndexError
```

❗ **IndexError: list index out of range**

len(test) は **4** なので有効なインデックスは **-4 ~ 3** まで。 **test[4]** や **test[-5]** はその範囲外なのでエラー。

10

リストの長さと `range` による走査

- `len(list)` でリストの**要素数**を取得できる
- `range(N)` は `0, 1, 2, ..., N-1` を返す（list ではなく range 型）
→ `list(range(N))` で list に変換可能
- `range(len(list))` と組み合わせて走査するパターンをよく使う

```
1 student_scores = [85, 90, 92, 78, 92, 88]
2
3 print(len(student_scores))           # 6
4 print(list(range(len(student_scores)))) # [0, 1, 2, 3, 4, 5]
5
6 for i in range(len(student_scores)):
7     print(f"出席番号{i+1}は、{student_scores[i]}点です。")
```

出席番号1は、85点です。
出席番号2は、90点です。
...

11

スライス（部分集合の取得）

リストの一部を切り取って別のリストにできる。

```
1 list[切り取る先頭インデックス : 切り取る末尾インデックス+1]
```

```
1 org_test = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
2 sliced_list = org_test[2:5]
3 print(sliced_list) # [2, 3, 4]
```

💡 ポイント

末尾インデックスは「取りたい最後の要素のインデックス+1」を指定する。

`org_test[2:5]` → インデックス 2, 3, 4 の要素を取得。

12

2次元配列

list の要素として list を入れると **2次元配列** になる。

アクセスは **`list[行][列]`** の形式。

```
1 list_of_list = [[1, 2, 3], [4, 5, 6]]
2 print(list_of_list)           # [[1, 2, 3], [4, 5, 6]]
3 print(list_of_list[0][1])     # 2
```

	列0	列1	列2
行0 →	[1,	2,	3]
行1 →	[4,	5,	6]

`list_of_list[0][1]` → 行0・列1 → **2**

13

N 次元配列

2 次元以上の多次元化も可能。

```
1 N_dimensional_list = [[[1, 2, 3], [4, 5, 6]],  
2                        [[7, 8, 9], [10, 11, 12]]]  
3  
4 print(N_dimensional_list[1][1][2]) # 12
```

- **[1]** → 2 番目のブロック **[[7, 8, 9], [10, 11, 12]]**
- **[1]** → その中の2番目 **[10, 11, 12]**
- **[2]** → その中の3番目 → **12**

14

list 同士の結合

+ 演算子でリスト同士を結合できる。

```
1 list1 = ["banana", "apple"]  
2 list2 = ["orange", "peach"]  
3  
4 print(list1 + list2) # ['banana', 'apple', 'orange', 'peach']  
5  
6 # += 演算も可能  
7 list1 += list2  
8 print(list1) # ['banana', 'apple', 'orange', 'peach']
```

① 重複はそのまま追加される

同じ要素があっても重複して追加される（重複排除はされない）。

```
1 list3 = ["banana", "grape"]  
2 print(list1 + list3)  
3 # ['banana', 'apple', 'orange', 'peach', 'banana', 'grape']
```

15

ソート

sort() と **sorted()** でリストを並び替えられる。

```
1 scores = [85, 90, 92, 78, 92, 88]
2
3 # sort(): 元のリストを直接変更 (ミュータブル!)
4 scores.sort()
5 print(scores) # [78, 85, 88, 90, 92, 92]
6
7 # sorted(): 新しいリストを返す (元は変わらない)
8 original = [85, 90, 92, 78]
9 result = sorted(original)
10 print(original) # [85, 90, 92, 78] ← 変わらない
11 print(result)   # [78, 85, 90, 92]
12
13 # 降順は reverse=True
14 scores.sort(reverse=True)
15 print(scores) # [92, 92, 90, 88, 85, 78]
```

Python

💡 sort vs sorted の使い分け

- 元のリストを壊してよい → **sort()** (速い)
- 元のリストを保持したい → **sorted()** (安全)

sort() が in-place で変更する点は、第4回で学んだミュータブルの性質そのもの。

16

リスト内包表記

for 文を1行に圧縮してリストを作る書き方

例えば、[0, 1, 4, 9, 16]のリストを作りたい時、

```
1 squares = []
2 for x in range(5):
3     squares.append(x * x)
4 print(squares) # [0, 1, 4, 9, 16]
```

Python

と書けるけど、長いので以下のように一撃でも書ける。それがリスト内包表記

```
1 # 基本形: [式 for 変数 in イテラブル]
2 squares = [x ** 2 for x in range(5)]
3 print(squares) # [0, 1, 4, 9, 16]
```

Python

17

リスト内包表記（条件付き）

例えば、0-99の整数うち3の倍数以外のリストを作る場合、以下のようにfor文に条件を書くことになる。

```
1
2 not_divisible_by_3s = []
3 for x in range(100):
4     if x % 3 == 0:
5         continue
6     not_divisible_by_3s.append(x)
7 print(not_divisible_by_3s)
```

Python

これをリスト内包表記だとうこう書ける。短い

```
1 # 条件付き:[式 for 変数 in イテラブル if 条件]
2 not_divisible_by_3s = [x for x in range(100) if x % 3 != 0]
3 print(not_divisible_by_3s)
```

Python

① ノート

リスト内包表記はAtCoderでよく使う（次スライド）

18

リスト内包表記の使いどころ

第1回 AtCoder の参考コードに出てきた謎の構文：

```
1 As = [int(x) for x in input().split()]
```

Python

これはリスト内包表記だった。**input().split()** で **["1", "2", "3"]** という文字列のリストを作り、各要素を **int(x)** で整数に変換しながら新しいリストを作っている。

for 文で書くと以下と同じ意味：

```
1 input_str = input().split() # split(" ")と同じ。つまり、空白区切りで分けてlist化
2 As = []
3 for x in input_str:
4     As.append(int(x))
```

Python

19

【参考】異なる型の混在

Python の list は数値・文字列など **異なる型を混在** させることができる。

```
1 print([10, "text", 10.5])      # [10, 'text', 10.5]
2 print([10, [1, 2, 3], "text"]) # [10, [1, 2, 3], 'text']
3
4 test = [10, [1, 2, 3], "text"]
5 print(test[2])                 # text
6 print(test[1][2])              # 3
```

Python

💡 実際には使わない

管理しにくいため実務では使わない。ただし **多次元配列** はこの仕組みを応用している。

20

まとめ

- インデックスは **ゼロ始まり**、マイナスで末尾からアクセスできる
- **append** で末尾追加、**in** で存在確認、**sort** / **sorted** で並び替え
- スライス **[i:j]** ・多次元配列・**+** 結合など、組み合わせ次第で幅広く使える
- リスト内包表記 **[式 for x in ...]** で **for** + **append** を1行に圧縮できる
- **list** はミュータブル — コピーが必要なときは **.copy()** を使う

21

演習問題



```
1 scores = [85, 90, 92, 78, 92, 88, 65, 100, 73, 80]
```

Python

上記のリストをコピーして、以下の操作を実装せよ

1. **range(len(...))** を使って、**出席番号（1始まり）と点数**を表示
2. **スライス** を使って、上位5名分（先頭5つ）と下位5名分（末尾5つ）をそれぞれ別のリストに分けて表示
3. **sorted** を使って、**元のリストを変えずに**降順に並び替えた新しいリストを作り、**上位3名の点数**を表示

▶ 解答例を見る

22

演習問題（リスト内包表記）



以下の操作を実装せよ

1. `range(1, 11)` を使って、**1から10までの平方数**のリストを作ろう（`[1, 4, 9, ..., 100]`）
2. `scores` から、**80点以上の点数だけ**を取り出した新しいリストを作ろう
3. 標準入力で受け取った空白区切りの整数列を、**list of int** に変換しよう（AtCoder 頻出パターン）

▶ 解答例を見る

23

演習問題（2次元配列）



```
1 # 行：学生（0：A さん，1：B さん，2：C さん）
2 # 列：科目（0：国語，1：数学，2：英語）
3 table = [
4     [80, 90, 70],
5     [60, 85, 95],
6     [75, 70, 80],
7 ]
```

Python

上記の学生の国語・数学・英語の点数をもつ2次元配列をコピーして、以下の操作を実装せよ。

1. **B さんの数学**の点数を表示しよう
2. **C さんの3科目合計**を表示しよう
3. **各学生の平均点**を `for` 文で順に表示しよう

▶ 解答例を見る

24